

1 Creating and Populating Tables

- Point 1
- Point 2

2 Simple Queries

- Point 1
- Point 2

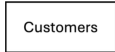
3 Aggregate Queries

- Point 1
- Point 2

4 Entity-Relationship Model

4.1 Entity Sets

- Identifiable things grouped into sets
- Diagram: named box
- All entities in set have same attributes, different values

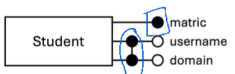


4.2 Attributes

- Value-oriented: atomic types (INTEGER, TEXT, etc.)
- Diagram: circle with name outside
- Simple: single attribute $\Rightarrow$ circle
- Derived: computed from others $\Rightarrow$ dashed line (e.g., age from date_of_birth)

4.3 Identifiers

- 1 or more attributes identify entity uniquely
- Single: black circle on attribute
- Composite: connected black circles (e.g., name + version)
- Candidate Key: all minimal sets identifying entity
- Property: ≥ 1 candidate key per entity set
- Minimality condition: cannot remove any attribute without losing identification
- Minimal $\neq$ minimum (local vs. global)



Relationships

4.4 Relationship Sets

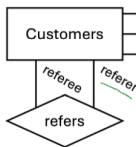
- Associates ≥ 2 entities (typically 2)
- No duplicates: set property
- Diagram: named diamond
- Description uses verbs (download, refers, supervise)

4.5 Relationship Attributes

- Not distinguished by own attributes
- Identified by: participating entities' identifiers
- Dependent on entities $\Rightarrow$ attributes are entity-dependent

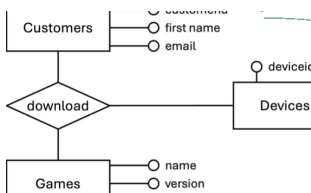
4.6 Same Entity Set Relationships

- Associates entities from same set
- Participation roles: named labels (e.g., referee $\leftrightarrow$ referrer)
- Rule: participation can always be named even across different entity sets



4.7 n-ary Relationships

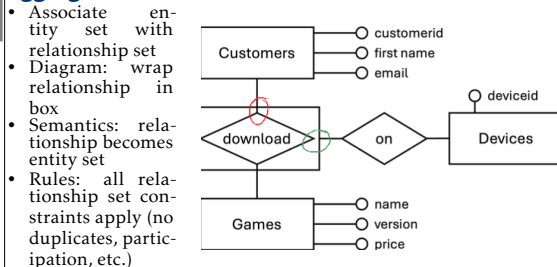
- Relate ≥ 2 entity sets
- All participation mandatory: no NULL allowed
- Still a set: no duplicates



4.8 Relationship Identifiers

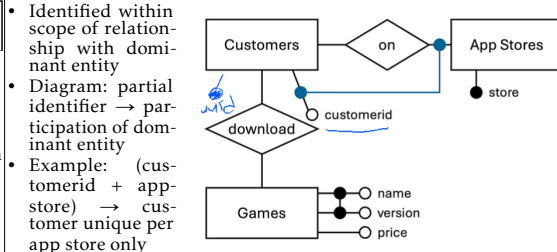
- Identified by: participating entities only
- Rule: NO black circle on relationship attribute
- Invalid: black circle on deviceid $\Rightarrow$ violates uniqueness (duplicate downloads)
- Solution: promote attribute to entity set (Devices) $\Rightarrow$ allows multiple same-game downloads per device

Aggregation



Weak Entity Sets

4.9 Partial Identification



4.10 Terminology

- Weak entity set: ≥ 1 partial key
- Identifying relationship: enables weak entity identification
- Dominant entity set: partner in identifying relationship
- Variant: can have both partial and non-partial keys
- Variant: partial key can have multiple dominant entities (uncommon)

Participation Constraints

4.11 Cardinality Format

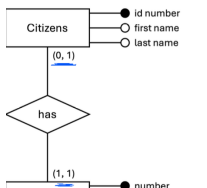
- (min, max) between entity set and relationship
- min = 0 (optional) or 1 (mandatory)
- max = 1 (one) or n (many)

4.12 Classifications (Binary Relations)

- (1, $_$) $\Rightarrow$ mandatory participation
- (0, $_$) $\Rightarrow$ optional participation
- ($_$, 1) on both sides $\Rightarrow$ one-to-one
- ($_$, 1) on one, ($_$, n) on other $\Rightarrow$ one-to-many
- ($_$, n) on both sides $\Rightarrow$ many-to-many
- Default (omitted): (0, n)

4.13 Examples

- Citizens (0,1) has Passports (1,1): one citizen $\Rightarrow$ 0-1 passports, one passport $\Rightarrow$ exactly 1 citizen (one-to-one, optional)



Conversion to Relational Schema

4.14 Rule #1: Value Sets

- Value sets $\rightarrow$ domains
- ER attributes $\rightarrow$ relation attributes + meaningful type
- Default: NOT NULL constraint
- Justify every NULL instance (avoid unless necessary)

4.15 Rule #2: Entity Sets

- Entity sets $\rightarrow$ relations
- Attributes $\rightarrow$ relation attributes
- Candidate keys $\rightarrow$ PRIMARY KEY + UNIQUE NOT NULL

5 Nested Queries

- Point 1
- Point 2

6 Relational Algebra

6.1 Relational Algebra Operators

Unary Operation	Symbol	Set Operation	Symbol
Selection	σ	Union	$\cup$
Projection	π	Intersection	$\cap$
Renaming	ρ	Set Difference	$-$
Binary Operation		Symbol	
Cross Product	$\times$		
Inner/Natural Join	$\bowtie$		
Outer Join	$\bowtie \ltimes \bowtie$		

6.2 Notation & Fundamentals

- σ Selection (WHERE)
- π Projection (SELECT)
- ρ Renaming (AS)
- $\times$ Cross Product
- $\bowtie$ Natural Join
- $\cup$ Union
- $\cap$ Intersection
- $-$ Set Difference

Properties: Sets (no duplicates) • No row order • Column order matters • Tuples immutable

6.3 Unary Operators

- Selection: $\sigma_c(R)$
- Filter rows by condition c
- Same columns, fewer rows
- Example: $\sigma_{country='SG'}(customers)$
- NULL: $\neg(NULL = NULL) \Rightarrow$ row excluded

Projection: $\pi_{[list]}(R)$

- Keep columns in specified order
- Auto-deduplicates (set semantics)
- Cannot add/derive columns
- Example: $\pi_{[name, email]}(customers)$
- $\neq$ SQL SELECT (requires DISTINCT)
- Renaming: $\rho(R_1, R_2)$ or $\rho(R, R_2(A \rightarrow B))$
- Rename relation & attributes locally
- For self-joins: $\rho(downloads, d_1)$
- Scope: within query only

6.4 Set Operations

Requirement: Union-compatible (same column types/count)

- $R \cup S$ All rows (dedup)
- $R \cap S$ Only common rows
- $R - S$ Rows in R not in S

6.5 Binary Operators

- Cross Product: $R_1 \times R_2$
- Combine each row of R_1 with each row of R_2
- Output rows: $n \times m$
- Usually used with selection: $\sigma_c(R_1 \times R_2)$
- Expensive for large tables

Inner Join: $R_1 \bowtie_c R_2$

- Equivalent: $\sigma_c(R_1 \times R_2)$
- Only matching rows
- Example: $R_1 \bowtie_{R_1.id=R_2.id} R_2$
- NULL in join key $\Rightarrow$ no match

Natural Join: $R_1 \bowtie R_2$

- Join on common attribute names
- Keeps one copy of common columns
- Common attrs first, then R_1 uniques, then R_2 uniques
- Pitfall: ambiguous column names

Outer Joins

- Left ($\ltimes$): Keep all left rows, NULL for unmatched right
- Right ($\rtimes$): Keep all right rows, NULL for unmatched left
- Full ($\bowtie \ltimes \bowtie$): Keep all rows both sides

6.6 Query Optimization

Equivalences:

1. Push selection down: $\sigma_c(\pi[cols](R)) = \pi[cols](\sigma_c(R))$
2. Split AND: $\sigma_{A \wedge B}(R) = \sigma_A(\sigma_B(R))$
3. Cannot split OR: $\sigma_{A \vee B}$ stays together
4. Reorder joins: Based on table size & selectivity

Strategy:

- Filter early (reduce rows)
- Cheapest conditions first
- Use explicit joins over cross product

6.7 NULL Handling

- NULL = NULL FALSE (not TRUE)
- IS NULL Test for NULL
- IS NOT NULL Test not NULL
- IS DISTINCT FROM Compare including NULL
- NULL AND TRUE NULL
- NULL OR TRUE TRUE

In selection: condition must be TRUE • NULL $\Rightarrow$ row excluded

6.8 SQL $\leftrightarrow$ RA Translation

Pipeline: Parse SQL $\rightarrow$ RA expression $\rightarrow$ Optimize $\rightarrow$ Execute

- WHERE σ
- SELECT cols π
- Mapping: SELECT DISTINCT π (auto in RA)
- FROM R1, R2 $\times$ or $\bowtie$
- UNION $\cup$

6.9 Common Pitfalls & Fixes

1. Projection hides duplicates: $\pi_{[name]}$ removes duplicate names $\Rightarrow$ use if intentional
2. Natural join ambiguity: Same column name, different meaning $\Rightarrow$ use ρ or explicit join
3. NULL in joins: NULL keys never match $\Rightarrow$ use outer joins for unmatched
4. Forgot DISTINCT in SQL: SQL RA does auto-dedup, SQL needs explicit $\Rightarrow$ add DISTINCT
5. Order matters in projection: Column order in $\pi_{[list]}$ is output order

6.10 Complex Example

Find emails of SG customers who bought games \$5+:

$\pi_{[c.email]} \left(\sigma_{[c.country='SG' \wedge g.price>5]} \left((customers \bowtie_{c.id=d.cid} downloads) \bowtie_{d.gname=g.name} games \right) \right)$

Steps: 1. Join tables 2. Filter by conditions 3. Project email

6.11 Set Semantics vs SQL

- | Property | RA | SQL |
|------------------|--------------|---------------------|
| Duplicates | None (set) | Allowed (multi-set) |
| Row order | Undefined | Preservable |
| Projection dedup | Auto | Need DISTINCT |
| Equivalence | Formal rules | Query-dependent |

$\Rightarrow$ SQL: performance + DISTINCT cost. RA: pure sets.

Schema Tracking

$R(A_1, \dots, A_n)$ has schema

- $\sigma_c(R) \Rightarrow$ schema unchanged
- $\pi_{A_1, \dots, A_k}(R) \Rightarrow$ output = listed attributes only
- $\rho \Rightarrow$ renames attributes/relations
- $R \times S \Rightarrow$ schema = concatenated
- $R \bowtie S \Rightarrow$ schema = union of attributes

Projection Validity

- $A \notin \text{schema}(R) \Rightarrow \pi_A(R)$ invalid

Union Compatibility

- $R \cup S, R \cap S, R - S$ require:
- Same arity: same number of attributes
- Type match: corresponding attribute types match

Join Validity

- Natural join: shared attribute names required
- Theta join: condition attributes must exist
- After join: recompute schema before projection

Cross Product Rule

- Always defined, may create duplicate attribute names
- Often requires renaming before further operations

Exam Checklist

- Track schema after every operator
- Check projection attributes exist
- Check union arity match
- Validate join attributes exist

7 Stored Procedures and Functions

Stored Procedures & Functions

7.1 Function Syntax

```
CREATE OR REPLACE FUNCTION name(param TYPE, ...)
RETURNS TYPE AS $$
DECLARE
  var TYPE;
BEGIN
  -- code
  RETURN value;
END;
$$ LANGUAGE plpgsql;
```

```
7.2 Key Operators
• := assignment (NOT =)
• = comparison
• $$ dollar-quoted strings (or $tag$...$tag$)

7.3 Variable Declaration
DECLARE
  years INTEGER;
  price NUMERIC;
  name VARCHAR(32);

7.4 Control Flow
IF condition THEN
  -- statements
ELSIF condition THEN
  -- statements
ELSE
  -- statements
END IF;

7.5 Loops
WHILE condition LOOP
  -- statements
END LOOP;

7.6 Return Types
Scalar:
RETURNS INTEGER AS $$
BEGIN
  RETURN value;
END;

Single Record (OUT params):
FUNCTION f(OUT col1 TYPE, OUT col2 TYPE)
RETURNS RECORD AS $$
  SELECT ...; -- assigns to OUT params
Set of Rows:
RETURNS SETOF tablename AS $$
  SELECT * FROM tablename;
Explicit Table:
RETURNS TABLE(col1 TYPE, col2 TYPE) AS $$
  SELECT ...;

7.7 Parameters
FUNCTION f(IN x INT,      -- input only (default)
          OUT y INT,     -- output only
          INOUT z INT) -- both input & output

7.8 Procedures (No Return)
CREATE OR REPLACE PROCEDURE name(param TYPE, ...)
AS $$
DECLARE
  var TYPE;
BEGIN
  -- action code, no RETURN
END;
$$ LANGUAGE plpgsql;

CALL name(args); -- invoke

7.9 Error Handling
RAISE EXCEPTION 'message: %', var;
RAISE WARNING 'message';
RAISE NOTICE 'message';
RAISE INFO 'message';
RAISE LOG 'message';
RAISE DEBUG 'message';

7.10 Cursors
Declare:
DECLARE
  cur CURSOR (param TYPE) FOR
  SELECT ... WHERE ... = param;
Open:
OPEN cur(param := value);
Fetch & Loop:
LOOP
  FETCH [NEXT|PRIOR|FIRST|LAST|ABSOLUTE n|RELATIVE n]
  FROM cur INTO var;
  EXIT WHEN NOT FOUND;
  -- process var
END LOOP;
Close:
CLOSE cur; -- must close to avoid memory leak

7.11 Cursor Example: Max Price Jump
CREATE OR REPLACE FUNCTION max_increase(gname VARCHAR)
RETURNS NUMERIC AS $$
DECLARE
  cur CURSOR (vname VARCHAR) FOR
  SELECT g.price FROM games g
  WHERE g.name = vname ORDER BY g.version;
  res NUMERIC := 0;
  prev NUMERIC := 0;
  curr NUMERIC;
BEGIN
  OPEN cur(vname := gname);
  LOOP
```

```
  FETCH cur INTO curr;
  EXIT WHEN NOT FOUND;
  res := GREATEST(res, curr - prev);
  prev := curr;
END LOOP;
CLOSE cur;
RETURN res;
END;
$$ LANGUAGE plpgsql;

7.12 INTO Clause (Query → Variable)
SELECT col INTO var FROM table WHERE condition;
-- assigns result to var

7.13 Conditional Insert Example
IF NOT EXISTS (
  SELECT 1 FROM table WHERE condition
) THEN
  INSERT INTO table VALUES (...);
END IF;

7.14 Quick Reference: Cursor Navigation
FETCH NEXT      -- default, go forward
FETCH PRIOR     -- go backward
FETCH FIRST     -- jump to start
FETCH LAST      -- jump to end
FETCH ABSOLUTE n -- jump to index n
FETCH RELATIVE n -- move by n rows
Note: After OPEN, cursor is "before" first row.

8 Triggers

8.1 Trigger Template
% LEGEND: (optionals), choice/other
CREATE (CONSTRAINT) TRIGGER trigger_name
BEFORE/AFTER INSERT/UPDATE/DELETE OR (INSERT/UPDATE/DELETE)
(DEFERRABLE INITIALLY DEFERRED/INITIALLY IMMEDIATE)
FOR EACH ROW/FOR EACH STATEMENT
[WHEN (condition)] % row-level only
[REFERENCING NEW TABLE AS newrows] % statement-level only
EXECUTE FUNCTION function_name(arg,...);
• (CONSTRAINT): makes trigger a constraint trigger; supports DEFERRABLE semantics.
• BEFORE vs AFTER: BEFORE fires before the change (can modify/stop it); AFTER fires after change (sees final state).
• ROW vs STATEMENT: ROW runs per row (access NEW/OLD); STATEMENT runs once per statement (use transition tables).
• WHEN: allowed for row-level triggers (evaluated per row).
• REFERENCING ... TABLE AS: available for statement-level triggers to access all changed rows.
• DEFERRABLE INITIALLY DEFERRED/IMMEDIATE: controls whether constraint trigger execution can be delayed to commit.
• Constraint triggers must be AFTER, or else what?

8.2 Example
CREATE CONSTRAINT TRIGGER trg_complex
AFTER INSERT OR UPDATE OR DELETE ON Scores
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
WHEN (TG_OP = 'UPDATE' AND NEW.score < 60)
EXECUTE FUNCTION scores_check_func();
CREATE TRIGGER t
BEFORE INSERT ON Scores
FOR EACH ROW
EXECUTE FUNCTION f('min', '10');
-- inside f():
-- row-level: use NEW.score, OLD.*, TG_OP
-- literals: use TG_ARGV[0] -> 'min', TG_ARGV[1] -> '10'
=====
CREATE TRIGGER t_stmt
AFTER INSERT ON Scores
FOR EACH STATEMENT
REFERENCING NEW TABLE AS newrows
EXECUTE FUNCTION f_stmt();
-- statement-level:
-- inside f_stmt(): SELECT COUNT(*) FROM newrows;

9 Functional Dependencies

9.1 Algorithm for finding minimal keys
1. Definition: A key is a minimal set of attributes that functionally determines all attributes of the relation.
2. Given: a relation  $T(A, B, C, \dots)$  and a set of functional dependencies  $\mathcal{F}$  on  $T$ .
3. Algorithm:
  (a) Get undervivable attributes  $U$ 
  (b) Find minimal keys. Start with  $U+$ , then  $UX+$ , then  $UXY+$ , etc.
  (c) All subsets with closures containing every attribute of  $R$  are superkeys.
  (d) From the superkeys, choose those minimal under set inclusion; these are the keys.
```

```
9.2 Algorithm to find if FD is preserved
1. Find Projection of  $S$  on each Fragment (e.g.  $R_1 = A, B$ )
  (a) List all subsets of fragment ( $R_1: A, B, AB$ )
  (b) Find closure for each subset using original  $S$  ( $R_1: A+, B+, AB+$ )
  (c) Delete RHS attributes NOT in fragment (e.g.  $R_1$  has  $A+=ABCD$  -remove  $CD \rightarrow A+=AB$ )
  (d) Delete trivial attributes on RHS ( $A+=AB$  becomes  $A \rightarrow B$  |  $A \rightarrow A$  or  $AB \rightarrow AB$  is entirely removed)
  (e) Set of non-empty closures form the projection  $S_1$  on fragment  $R_1$ 
2. Union all projected FDs ( $S_1$  U  $S_2$  U ...) to get  $S_u$ 
3. Check if FD in  $S$  can be derived from  $S_u$ 

9.3 Lossless Join
• Decomposition  $R \rightarrow R_1, R_2$  is lossless iff  $(R_1 \cap R_2) \rightarrow R_1$  or  $(R_1 \cap R_2) \rightarrow R_2$ .
• I.e., the common attributes form a superkey of  $R_1$  or  $R_2$ .
• The lossless test uses the original FD closure of  $R$ .

10 BCNF

10.1 BCNF: Definition
A relation  $R$  is in Boyce-Codd Normal Form (BCNF) if every non-trivial and decomposed functional dependency (FD) has a superkey on its left-hand side.

10.2 BCNF Decomposition Algorithm
1. Pick a FD ( $X \rightarrow Y$ ) that violates the BCNF property.
2. Fragment 1 contains all attributes in  $X^+$ .
3. Fragment 2 contains all attributes in  $R - (X^+ - X)$ .
4. Repeat steps 1–3 on any fragment that still contains a functional dependency violating BCNF.

11 3NF

11.1 Third Normal Form (3NF)
A table satisfies 3NF if and only if, for every non-trivial FD  $X \rightarrow Y$ :
1.  $X$  is a superkey, OR
2. Every attribute in  $Y$  is a prime attribute (i.e., appears in some candidate key).

11.2 Minimal Basis
 $\Sigma_{\min}$  is a minimal basis for  $\Sigma$  if:
1. Every FD in  $\Sigma_{\min}$  can be derived from  $\Sigma$  and vice versa.
2. Every FD in  $\Sigma_{\min}$  is non-trivial and has a single attribute on the right-hand side.
3. No FD in  $\Sigma_{\min}$  is redundant.
4. For each FD in  $\Sigma_{\min}$ , no attribute on the left-hand side is redundant.

11.3 3NF Decomposition Algorithm
1. Minimal Basis. Derive a minimal basis of  $S$ .
2. Combine RHS. In the minimal basis, combine FDs that have the same left-hand side.
3. Create Relations. Create a relation (table) for each FD remaining.
4. Ensure Key in Some Relation. If none of the relations contain a key of the original relation  $R$ , create an additional relation that contains a key of  $R$ .
5. Remove Subsumed Relations. Remove subsumed relations (i.e., relations that are subsets of others).

11.4 Algorithm for Minimal Basis
1. Split RHS. Split up FDs so RHS only have 1 attribute.
2. Minimize LHS. Remove redundant attributes from the LHS of each FD. ( $AB \rightarrow Y$ : Remove  $A$  then see if  $B \rightarrow Y$  is implied by the original  $S$  (including  $AB \rightarrow Y$ ), yes means can remove)
3. Remove FDs. Remove redundant FDs. (Remove FD  $X \rightarrow Y$ . If  $X \rightarrow Y$  is still implied by  $S$  means can remove.)
```